

Proyecto final de Optimización Convexa

Optimización de hiperparámetros en modelos predictivos

Acoyani Garrido Sandoval

19 de mayo de 2026

1. Introducción

El objetivo del presente proyecto final es aplicar los conceptos aprendidos en el curso de Optimización Convexa para resolver un problema de optimización de hiperparámetros en un modelo predictivo.

En general, los pasos del proyecto son:

1. Seleccionar un problema de aprendizaje supervisado o no supervisado. Puede ser uno que esté siendo desarrollado, o que tenga relevancia personal o académica.
2. Elegir modelos base que tengan hiperparámetros susceptibles de ser optimizados.
3. Identificar los hiperparámetros críticos del modelo y definir un espacio de búsqueda adecuado para cada uno de ellos.
4. Plantear el problema de buscar los valores de los hiperparámetros como un problema de optimización: regularización L1, L2 o *elasticnet*, conjuntos convexos y restricciones, minimización de funciones convexas, dualidad y condiciones de optimalidad, o métodos de optimización tales como gradiente descendente.
5. Implementar la solución del problema en una Jupyter notebook. Está permitido el uso de librerías, siempre y cuando se justifique el método de optimización.

2. Selección de conjunto de datos

El conjunto de datos usado para estos fines es el **Telco Customer Churn** (Kaggle, 2018), un conjunto de datos desarrollado por IBM originalmente como una muestra de datos incluída con el software **IBM Cognos Analytics Server** (IBM, 2026), un software de análisis y visualización de datos basado en Java Enterprise Edition. Este conjunto de datos reviste una doble importancia personal para mí, ya que por una parte se trata de un conjunto de datos relacionado con comportamiento de clientes, giro en el cual actualmente manejo un negocio denominado **BuzónQR®**; y por otro lado, por el hecho de ser un conjunto de datos inicialmente concebido para probar las capacidades del software IBM Cognos Analytics Server, una plataforma de inteligencia de negocios inicialmente desarrollada por la empresa canadiense Cognos que fue comprada por IBM hacia el año 2008, orientada principalmente a visualización, procesamiento, análisis y exploración de datos; mi primer trabajo, el cual desempeñé en IBM desde 2012 hasta 2018, fue como administrador de una serie de instancias de dicha plataforma.

2.1. Características del conjunto de datos

El conjunto de datos *Telco Customer Churn* tiene como finalidad ofrecer un medio para mejorar la retención de clientes a través de información acerca de ellos y un dato que muestra si han dejado la empresa de telefonía en el último mes.

En este conjunto de datos, cada fila representa uno de los usuarios de la empresa de telefonía, y cada columna representa información acerca del cliente. Las características de este modelo son:

1. **Variable dependiente:** *Churn*. Es una variable binaria cuyos valores son “Yes” o “No”, e indica si el cliente ha dejado la compañía en el último mes. La distribución de clases de esta variable es de 73.5 % retenidos ($\Pr(Churn = No)$) y 26.5 % no retenidos ($\Pr(Churn = Yes)$); está moderadamente desbalanceado, lo cual es importante a la hora de elegir hiperparámetros de ponderación de clase.
2. **Características:** Encuadran en 4 grupos naturales: **demográficas** (*gender*, *SeniorCitizen*, *Partner*, *Dependents*), **información de cuenta** (*tenure* (meses como cliente), *Contract* (mensual, anual, bienal), *PaperlessBilling*, *PaymentMethod*), y **servicios suscritos** (*PhoneService*, *MultipleLines*, *InternetService*, *OnlineSecurity*, *OnlineBackup*, *DeviceProtection*, *TechSupport*, *StreamingTV*, *StreamingMovies*), y **cargos** (*MonthlyCharges*, *TotalCharges*).

Si realizamos un proceso de *one-hot encoding* de las características (descrito más adelante), obtenemos un total de 30 dimensiones de datos; lo cual implica que será necesario elegir características a través de la regularización, a la

vez que la cantidad de características es lo suficientemente pequeña para que un solver convexo lo pueda resolver.

En general, podemos observar que se trata de un problema de clasificación supervisada binaria, lo cual hace que el modelo natural a elegir sea **regresión logística**. Además de prestarse de forma natural a variables dependientes binarias, este modelo tiene 3 características que van en línea con el contenido del curso:

1. La regresión logística tiene una función de log-verosimilitud negativa convexa en parámetros, lo que hace que la superficie de pérdida tenga un mínimo global único.
2. Añadir regularización L1, L2 o elasticnet preservará la convexidad, lo que nos permitirá aplicarlas sin complejidades.
3. El problema de optimización tiene un planteamiento dual y condiciones KKT bien conocidas.

2.2. One-hot encoding de características

En la subsección anterior, mencioné que un proceso de *one-hot encoding* de las características arrojó 30 dimensiones de datos. El proceso para hacer eso y las implicaciones se explican a continuación.

¿Qué es? El *one-hot encoding* (codificación en caliente) es una técnica de preprocesamiento que convierte variables categóricas en variables numéricas binarias. Para cada variable categórica con k categorías distintas, se crean k columnas binarias (indicadoras), una por categoría, donde el valor 1 indica que la observación pertenece a esa categoría y 0 en caso contrario. En la práctica, para evitar multicolinealidad perfecta (*dummy variable trap*), se elimina una de las k columnas por variable, resultando en $k - 1$ columnas nuevas; la categoría eliminada queda implícitamente representada cuando todas las demás son 0.

Proceso El conjunto de datos *Telco Customer Churn* contiene 19 características (excluyendo *customerID* y la variable dependiente *Churn*). De ellas, 3 son ya numéricas continuas (*tenure*, *MonthlyCharges*, *TotalCharges*) y 1 es binaria numérica (*SeniorCitizen*), por lo que se mantienen sin transformación. Las 15 características categóricas restantes se procesan con `pd.get_dummies(..., drop_first=True)` de la librería *Pandas*, eliminando una categoría por variable para evitar la trampa de la variable ficticia. La tabla siguiente resume la expansión de dimensiones:

Característica	Categorías	Columnas generadas
<i>gender</i>	Female, Male	1
<i>Partner</i>	No, Yes	1
<i>Dependents</i>	No, Yes	1
<i>PhoneService</i>	No, Yes	1
<i>PaperlessBilling</i>	No, Yes	1
<i>MultipleLines</i>	No, No phone service, Yes	2
<i>InternetService</i>	DSL, Fiber optic, No	2
<i>OnlineSecurity</i>	No, No internet service, Yes	2
<i>OnlineBackup</i>	No, No internet service, Yes	2
<i>DeviceProtection</i>	No, No internet service, Yes	2
<i>TechSupport</i>	No, No internet service, Yes	2
<i>StreamingTV</i>	No, No internet service, Yes	2
<i>StreamingMovies</i>	No, No internet service, Yes	2
<i>Contract</i>	Month-to-month, One year, Two year	2
<i>PaymentMethod</i>	Bank transfer, Credit card,	3
	Electronic check, Mailed check	3
4 numéricas	(<i>SeniorCitizen</i> , <i>tenure</i> ,)	4
	(<i>MonthlyCharges</i> , <i>TotalCharges</i>)	4
Total		30

Implicaciones

1. **Dimensionalidad manejable.** Las 30 dimensiones resultantes son suficientemente pocas para que un solver convexo (como `liblinear` o `lbfgs` dentro de `sklearn.linear_model.LogisticRegression`) las maneje eficientemente.
2. **Necesidad de regularización.** Con 30 características y 7,043 observaciones, la razón de observaciones por característica es de aproximadamente 235:1, lo cual en principio es favorable. Sin embargo, varios subgrupos de variables indicadoras están correlacionados entre sí (p. ej., los servicios que dependen de *InternetService*), por lo que la regularización L1 o L2 sigue siendo conveniente para controlar la varianza del modelo y producir coeficientes interpretables.
3. **Interpretabilidad de coeficientes.** Cada coeficiente β_j en la regresión logística representa el cambio en el log-odds de *Churn* al pasar de la categoría de referencia a la categoría representada, manteniendo las demás características constantes; esto facilita la interpretación de resultados.
4. **Escala.** Las columnas numéricas continuas (*tenure*, *MonthlyCharges*, *TotalCharges*) tienen escalas distintas a las variables indicadoras $\{0, 1\}$, por lo que es necesario estandarizarlas (media cero, varianza unitaria) antes de ajustar el modelo con regularización, para que la penalización afecte a

todos los coeficientes de forma equitativa.

3. Desarrollo del problema

Habiendo expuesto la composición de nuestro conjunto de datos, la metodología que seguiremos será:

1. **Planteamiento del problema:** Predecir $\Pr(\text{Churn} = 1|x)$ a partir de las características de los clientes.
2. **Modelos:** Regresión logística como modelo primario, SVM lineal como modelo secundario. Compararé la pérdida logística contra la *hinge loss*. Privilegiaré el uso de modelos lineales con el fin de mantenernos en un marco de referencia convexo.
3. Plantear los hiperparámetros a optimizar y su espacio de búsqueda.
4. Plantear un problema de optimización convexa.
5. Implementación.

4. Modelo predictivo de regresión logística

4.1. Planteamiento del problema de optimización

4.1.1. Objetivo predictivo

Dado un vector de características $\mathbf{x} \in \mathbb{R}^{30}$ que describe a un cliente, queremos estimar:

$$\Pr(\text{Churn} = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \quad (1)$$

donde $\sigma(\cdot)$ es la función sigmoide, $\mathbf{w} \in \mathbb{R}^{30}$ es el vector de pesos que cada característica tiene en el resultado final, $\mathbf{x}$ es el vector de características leídas del conjunto de datos, y $b \in \mathbb{R}$ es el sesgo. La sigmoide es el fundamento de la **regresión logística**: si asumimos que el log-odds de la probabilidad es lineal en $\mathbf{x}$,

$$\log \frac{\Pr(\text{Churn} = 1 \mid \mathbf{x})}{\Pr(\text{Churn} = 0 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x} + b, \quad (2)$$

despejar $\Pr(\text{Churn} = 1 \mid \mathbf{x})$ da exactamente $\sigma(\mathbf{w}^\top \mathbf{x} + b)$. Cada componente w_j del vector $\mathbf{w}$ es la pendiente asociada a la j -ésima característica, análoga al coeficiente de una regresión lineal.

4.1.2. Función de pérdida (log-verosimilitud negativa)

Para un conjunto de entrenamiento $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ con $y_i \in \{0, 1\}$, la pérdida logística es:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] \quad (3)$$

donde $\hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$. Esta función es **convexa en** $(\mathbf{w}, b)$, lo que garantiza la existencia de un mínimo global único.

4.1.3. Problema de optimización con regularización

Para controlar la varianza del modelo y forzar escasez o suavidad en los coeficientes, añadimos regularización:

$$\min_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b) + \lambda \Omega(\mathbf{w}) \quad (4)$$

donde la penalización $\Omega(\mathbf{w})$ puede ser:

Regularización	$\Omega(\mathbf{w})$	Propiedad
L2 (Ridge)	$\frac{1}{2} \ \mathbf{w}\ _2^2$	Coefficientes pequeños, solución única
L1 (Lasso)	$\ \mathbf{w}\ _1$	Selección de características (<i>sparse</i>)
ElasticNet	$\alpha \ \mathbf{w}\ _1 + \frac{1-\alpha}{2} \ \mathbf{w}\ _2^2$	Combinación de ambas

Dado el desbalance de clases (73.5 % / 26.5 %), se añade también `class_weight='balanced'`, que pondera cada ejemplo i con $w_i \propto 1/\text{Pr}(y_i)$. El hiperparámetro principal a optimizar es $C = 1/\lambda$ (inverso de la intensidad de regularización): un C grande implica poca regularización; un C pequeño impone penalización fuerte y coeficientes más cercanos a cero.

4.2. Implementación

4.2.1. Preprocesamiento

Los pasos de preprocesamiento aplicados fueron:

1. Eliminar `customerID` (identificador sin valor predictivo).
2. Convertir `TotalCharges` a numérico; los espacios en blanco de clientes nuevos se imputan como 0.
3. Codificar `Churn` como variable binaria $\{0, 1\}$.
4. Aplicar *one-hot encoding* con `drop_first=True` a las variables categóricas.
5. Separar características (X) y variable dependiente (y).

6. Dividir en conjunto de entrenamiento (80 %) y prueba (20 %) con estratificación por clase.
7. Estandarizar las columnas numéricas continuas (*tenure*, *MonthlyCharges*, *TotalCharges*) con media 0 y varianza 1, ajustando el **StandardScaler** únicamente sobre el conjunto de entrenamiento para evitar fuga de información.

El resultado es un conjunto de entrenamiento de 5,634 observaciones y uno de prueba de 1,409, ambos con 30 características.

4.2.2. Modelo inicial y el solver L-BFGS

Se ajustó un primer modelo de regresión logística con regularización **L2**, $C = 1$ y `class_weight='balanced'`. El solver **lbfgs** (*Limited-memory Broyden-Fletcher-Goldfarb-Shanno*) minimiza internamente:

$$\min_{\mathbf{w}, b} \frac{1}{n} \sum_{i=1}^n [-y_i \log \sigma_i - (1 - y_i) \log(1 - \sigma_i)] + \frac{1}{2C} \|\mathbf{w}\|_2^2 \quad (5)$$

L-BFGS es un método de **quasi-Newton**: en lugar de calcular el Hessiano exacto $\nabla^2 \mathcal{L}$, lo aproxima iterativamente usando los últimos m pares $\{(\Delta \mathbf{w}_k, \Delta \nabla \mathcal{L}_k)\}$ para construir la dirección de descenso $\mathbf{d}_k = -H_k^{-1} \nabla \mathcal{L}(\mathbf{w}_k)$, seguida de *line search* para determinar el paso α_k :

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \mathbf{d}_k \quad (6)$$

Es el solver óptimo para este problema porque: (1) la convexidad estricta de la pérdida logística con L2 garantiza convergencia al mínimo global único; (2) la información de curvatura permite pasos más grandes que el gradiente descendente puro; (3) el costo de memoria $O(m \cdot p)$ es despreciable con $p = 30$; y (4) la regularización L2 es diferenciable en todo su dominio.

4.2.3. Evaluación del modelo base

Las métricas se calculan a partir de la matriz de confusión, que organiza las predicciones en cuatro celdas: Verdaderos Positivos (*VP*), Verdaderos Negativos (*VN*), Falsos Positivos (*FP*) y Falsos Negativos (*FN*). Las métricas son:

- **Precision** = $VP / (VP + FP)$: fracción de los predichos como “Abandona” que realmente abandonaron. Alta precisión implica pocas falsas alarmas.
- **Recall** = $VP / (VP + FN)$: fracción de los abandonos reales detectados por el modelo. Alto recall implica pocos abandonos sin detectar.
- **F1-score** = $2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$: media armónica entre ambas; útil con clases desbalanceadas.

- **Accuracy** = $(VP + VN)/n$: puede ser engañosa con desbalance (un clasificador que siempre predice “Retenido” tendría 73.5 % de accuracy).
- **Macro avg**: promedio simple entre clases, sin ponderar por tamaño; trata ambas clases como igualmente importantes.
- **Weighted avg**: promedio ponderado por el *support* de cada clase; refleja el desempeño global considerando el desbalance.

Para este problema, el **recall** de la clase positiva es la métrica de mayor peso de negocio: el costo de no detectar un cliente que abandona (falso negativo) supera al costo de una alarma innecesaria (falso positivo).

El **AUC-ROC** (*Area Under the Receiver Operating Characteristic Curve*) mide la capacidad discriminativa del modelo independientemente del umbral, graficando la tasa de verdaderos positivos contra la tasa de falsos positivos para cada umbral posible. Un valor de $AUC = x$ significa que, tomando al azar un par (cliente que abandona, cliente que no), el modelo asigna mayor probabilidad al primero en el $100x$ % de los casos. Un AUC de 0.5 equivale a un clasificador aleatorio; un AUC de 1.0 es un clasificador perfecto.

4.2.4. Validación cruzada del modelo base

Para obtener una estimación más robusta del desempeño se realizó una validación cruzada estratificada de 5 pliegues sobre el conjunto de entrenamiento. El procedimiento divide los datos en 5 subconjuntos manteniendo la proporción de clases; el modelo se entrena 5 veces usando 4 pliegues y evaluando en el restante. Esto reduce la varianza de la estimación y usa todos los datos para la evaluación. Los resultados fueron:

Métrica	Media	Desviación estándar
AUC-ROC	0.8454	± 0.0126
F1-score	0.6298	± 0.0220

Un AUC-ROC de 0.8454 indica que el modelo discrimina correctamente en el 84.5 % de los pares aleatorios. La desviación estándar baja en ambas métricas confirma que el desempeño es estable entre particiones. Estos valores constituyen la **línea base** contra la que se compararán los modelos optimizados; una mejora genuina deberá superar estos valores fuera del rango de ± 1 desviación estándar.

Referencias

- IBM (2026). Ibm cognos analytics. <https://www.ibm.com/products/cognos-analytics>. Consultado en Kaggle el 16 de Mayo de 2026.
- Kaggle (2018). Ibm telco customer churn dataset. <https://www.kaggle.com/datasets/blastchar/telco-customer-churn/data>. Consultado en Kaggle el 16 de Mayo de 2026.